B.Tech. Sem-V

Fundamentals of Operating System Design (CEUC301)

UNIT 4: CONCEPTUAL QUESTION BANK

Deadlock Modeling and System-level Handling

Unit	Unit 4
Question type	Conceptual, analytical and scenario-based

Syllabus coverage: Deadlock modelling; deadlock prevention and avoidance; detection and recovery trade-offs; deadlock versus starvation; resource-allocation trade-offs; starvation and livelock.

A. Deadlock Modelling and Necessary Conditions

1. Define deadlock in an operating system. How does it differ from a process that is merely delayed or waiting for a resource?
2. Explain the four necessary conditions for deadlock: mutual exclusion, hold and wait, no preemption, and circular wait.
3. Why are the four Coffman conditions necessary for deadlock in the classical resource-allocation model?
4. What is a resource-allocation graph? Explain how process nodes, resource nodes, request edges and assignment edges represent system state.
5. How can a cycle in a resource-allocation graph indicate deadlock when each resource type has a single instance?
6. Why does a cycle in a resource-allocation graph with multiple instances of resource types not always prove that deadlock exists?
7. Explain the difference between a resource request edge and a resource assignment edge in deadlock modelling.
8. How do reusable resources differ from consumable resources, and how can this distinction affect deadlock analysis?
9. Why is accurate modelling of resource instances, requests and allocations important before choosing a deadlock-handling strategy?

B. Deadlock Prevention and Avoidance

10. Explain the fundamental difference between deadlock prevention and deadlock avoidance.
11. How does eliminating the hold-and-wait condition prevent deadlock, and what resource-utilization or concurrency costs can result?
12. Explain how imposing a global ordering on resource types can prevent circular wait.
13. Why can allowing resource preemption help prevent deadlock for some resource types but be impractical for others?
14. How does requiring a process to request all resources before execution reduce deadlock risk, and why may this policy be inefficient?
15. What is a safe state? Explain why a safe state guarantees that there is some order in which all processes can complete.
16. Distinguish a safe state, an unsafe state and a deadlocked state. Why is an unsafe state not necessarily already deadlocked?
17. Explain the principle of the Banker's algorithm without performing numerical calculations. What information must the operating system know?

18. Why does deadlock avoidance depend on assumptions about maximum resource demands and the ability to predict future requests?
19. Compare prevention and avoidance in terms of runtime overhead, flexibility, resource utilization and required information.
20. A system has highly variable resource demands that are difficult to predict. Discuss whether prevention, avoidance or another handling strategy is likely to be more practical.

C. Deadlock Detection and Recovery Trade-offs

21. Explain the basic idea of allowing deadlocks to occur and detecting them later. When might this strategy be reasonable?
22. How does a deadlock detection algorithm differ conceptually from a deadlock avoidance algorithm?
23. Why is the frequency of deadlock detection an important system-design decision?
24. What are the trade-offs between frequent detection, which may reduce the duration of deadlock, and infrequent detection, which may reduce overhead?
25. Explain how terminating all deadlocked processes can recover the system. What information or work may be lost?
26. Compare terminating all deadlocked processes with terminating one process at a time until the deadlock is removed.
27. What factors should guide the choice of a victim process during deadlock recovery?
28. How can resource preemption be used to recover from deadlock, and why might rollback or checkpointing be required?
29. What is starvation in the context of repeated victim selection during deadlock recovery, and how can recovery policy reduce this risk?
30. Why must recovery strategies consider data consistency and application-level invariants, not only resource availability?
31. Compare prevention, avoidance, detection-and-recovery, and ignoring deadlocks in terms of overhead, predictability and suitability for different systems.

D. Deadlock versus Starvation and Resource Trade-offs

32. Differentiate deadlock from starvation in terms of system progress, resource ownership and the number of processes affected.
33. Can a system be free of deadlock and still be unfair to some processes? Explain using starvation.
34. How can a priority-based scheduler cause starvation even when resources are not involved in a circular wait?
35. Explain how resource allocation policies can create a trade-off between high utilization and fair access.
36. Why can aggressive resource reservation reduce contention while also reducing concurrency and throughput?
37. How can aging, fair queues or bounded-waiting policies reduce starvation, and what overheads might they introduce?
38. A low-priority process waits indefinitely while higher-priority processes continue to complete. Explain why this is starvation rather than deadlock.
39. Discuss how a system designer should balance fairness, throughput, resource utilization and predictable completion time.

E. Starvation and Livelock

40. Define livelock and distinguish it from deadlock and starvation.
41. How can two processes repeatedly respond to each other yet make no useful progress?
42. Explain how overly polite retry behaviour in a resource-acquisition protocol can produce livelock.

43. Why can a livelock system appear active because processes continue executing, even though useful work is not completed?
44. Compare the remedies for deadlock, starvation and livelock. Why is identifying the correct failure mode important?
45. How can randomized backoff, asymmetric rules or retry limits reduce livelock in competing processes?
46. What risks arise when retry mechanisms are introduced to solve contention without considering fairness or system-wide progress?
47. A distributed service repeatedly retries conflicting operations, but none completes. Explain how you would determine whether the problem is livelock, deadlock or starvation.

F. Integrative and Higher-Order Conceptual Questions

48. A database service uses locks for tables and records. Explain how resource-allocation modelling can help identify possible deadlocks and how lock ordering can reduce risk.
49. A system must guarantee predictable response times but has limited memory and device resources. Compare the suitability of prevention, avoidance and detection-and-recovery.
50. An operating system detects a deadlock but terminating a process could corrupt user data. Explain what recovery mechanisms and safeguards should be considered.
51. A resource manager avoids deadlock by reserving every requested resource in advance, but utilization falls sharply. Analyze the trade-off and suggest alternatives.
52. Design a conceptual monitoring plan for a system that experiences intermittent hangs. Identify evidence that would help distinguish deadlock, starvation and livelock.
53. A recovery mechanism repeatedly selects the same process as a victim. Explain the resulting fairness problem and propose a victim-selection policy that accounts for previous rollbacks.
54. Critically assess the statement: 'Deadlock prevention is always better than deadlock detection and recovery.' Use resource utilization, runtime overhead, predictability and recovery cost in your answer.

G. Deadlock Fundamentals, Resource Types and Conditions

55. Define deadlock and illustrate it with a realistic operating-system or application-level example.
56. Distinguish reusable resources from consumable resources. Give examples of each and explain how deadlock can arise in both cases.
57. Explain the four necessary conditions for deadlock and discuss why all four must hold simultaneously in the classical model.
58. Distinguish necessary conditions from sufficient conditions. Are the four Coffman conditions alone sufficient to conclude that a particular system is deadlocked? Explain.
59. Under what assumptions can a cycle in a resource-allocation graph be a sufficient condition for deadlock?
60. Explain how resource-allocation graphs model processes, resource types, requests and assignments.
61. How does the interpretation of a cycle change when resource types have multiple instances?
62. Describe a resource-allocation scenario in which a cycle exists but does not necessarily imply deadlock because a process can still finish and release resources.
63. Explain how each of the four deadlock conditions can be broken by a prevention strategy. What costs or limitations can arise from each strategy?
64. Why can a global ordering of resource types prevent circular wait, and what discipline must processes follow for this guarantee to hold?

H. Dining Philosophers Problem and Solutions

65. Describe the Dining Philosophers problem and explain how it models competition for shared resources.

66. How can the naive strategy of allowing each philosopher to pick up one fork and then wait for the other lead to deadlock?
67. Map the Dining Philosophers scenario to the four necessary conditions for deadlock.
68. Explain how limiting the number of philosophers allowed to compete for forks at once can prevent deadlock.
69. How can imposing an asymmetric fork-pickup order prevent circular wait in the Dining Philosophers problem?
70. Compare a semaphore-based solution and a monitor-based solution to the Dining Philosophers problem in terms of clarity and correctness.
71. Why must a correct Dining Philosophers solution address both deadlock freedom and the possibility of starvation?
72. What trade-offs arise between maximizing concurrency and simplifying synchronization in solutions to the Dining Philosophers problem?
73. A proposed solution prevents deadlock by allowing only one philosopher to eat at a time. Explain why it is correct in one respect but potentially inefficient.
74. Design criteria for evaluating a Dining Philosophers solution: identify the safety, progress, fairness and resource-utilization properties you would test.

I. Safe and Unsafe States; Banker's Algorithm

75. Define safe state and unsafe state in deadlock avoidance. Why does an unsafe state not necessarily mean that deadlock has already occurred?
76. What is a safe sequence, and how does it demonstrate that all processes can finish under a given resource-allocation state?
77. Explain the purpose of the Banker's algorithm and the assumptions it makes about maximum resource requirements.
78. What information is represented by the Available, Max, Allocation and Need data structures in the Banker's algorithm?
79. Explain conceptually how the Banker's safety algorithm determines whether a state is safe without carrying out a numerical example.
80. How does the resource-request algorithm decide whether a request can be granted immediately or should be delayed?
81. Why must the system tentatively allocate resources and run a safety check before granting a request under the Banker's algorithm?
82. What practical limitations arise when processes cannot declare maximum resource demands accurately in advance?
83. How can conservative avoidance decisions reduce resource utilization or delay processes even when resources are currently available?
84. A system rejects a request because it would lead to an unsafe state. Explain why this decision is not the same as declaring the requesting process deadlocked.
85. What additional information is needed to turn a conceptual description of the Banker's algorithm into a numerical safety-check exercise?

J. Detection, Recovery, Starvation and Livelock: Consolidation

86. Compare deadlock prevention, avoidance, detection-and-recovery and ignoring deadlocks as system-level policies.
87. How should a system designer choose the frequency of deadlock detection, considering runtime overhead and the cost of prolonged deadlock?
88. Compare process termination, resource preemption and rollback as deadlock-recovery techniques.

89. Which factors should guide victim selection during recovery, and how can repeatedly choosing the same victim cause starvation?
90. Explain how starvation can occur even if the system is not deadlocked, and identify policies that can improve fairness.
91. Differentiate deadlock, starvation and livelock by considering whether processes are blocked, whether useful work is completed, and whether system-wide progress occurs.
92. A pair of processes repeatedly releases and retries resources in response to each other, but neither completes. Explain why this may be livelock and propose remedies.
93. A system remains responsive overall, but one process waits indefinitely for a resource. Explain why this indicates starvation rather than system-wide deadlock.
94. Design a troubleshooting approach for a system hang that collects evidence to distinguish deadlock, starvation and livelock before selecting a remedy.